

COMPSCI5103 -- Deep Learning (M) --

Comprehensive Exam Notes

University of Glasgow | MSc Level | 2024-25 Lecturers: Dr Kevin Bryson (Weeks 1-5, Fundamentals) & Dr Ali Gooya (Weeks 6-10, Advanced Topics) **Exam Format:** Open book, 1.5 hours + 30 min additional time, answer ALL 3 questions, worth 60 marks total (60% of course) **Key Textbooks:** *Understanding Deep Learning* (Simon Prince), *Deep Learning* (Goodfellow, Bengio, Courville), *Deep Learning with PyTorch* (Stevens & Antiga)

Table of Contents

1. [Introduction to Deep Learning \(Week 1\)](#)
 2. [MLPs, SGD & Backpropagation \(Week 2\)](#)
 3. [Convolutional Neural Networks \(Week 3\)](#)
 4. [ML Workflow & Practical Aspects \(Week 4\)](#)
 5. [Optimisation, Generalisation, Regularisation \(Week 5\)](#)
 6. [Visualisation & Interpretability \(Week 6\)](#)
 7. [RNNs & Transformers \(Week 7\)](#)
 8. [Autoencoders \(Week 8\)](#)
 9. [VAEs & GANs \(Week 9\)](#)
 10. [Exam Strategy & Past Paper Analysis](#)
-

1. Introduction to Deep Learning (Week 1)

1.1 Historical Timeline

Year	Milestone
1943	Artificial neurons (McCullough-Pitts)
1958	Perceptron learning machine (Rosenblatt)
1959	Human visual cortex studies (Hubel-Wiesel)
1980	Neocognitron (Fukushima)
1986	Backpropagation for ANNs (Rumelhart, Hinton, Williams)
1989	LeNet -- first CNN (Yann LeCun et al.)
2012	AlexNet wins ImageNet by huge margin -- woke everyone up
2013	Atari Game Playing with Deep Reinforcement Learning
2014	GANs presented (Goodfellow et al.)
2016	AlphaGo beats world champion
2017	Transformer model -- "Attention is All You Need" (Google Brain)
2018	Turing Award to Bengio, Hinton, LeCun
2019	AlphaFold wins CASP13 protein folding
2022	ChatGPT released by OpenAI
2024	Nobel Prize to Hopfield and Hinton for DNNs

1.2 Why Deep Learning Became Successful

Six converging factors: 1. **Theory** -- Five decades of ML theory improvements (Bayesian estimation, bias/variance, VC dimension, Cybenko's theorem) 2. **Algorithms** -- Convolutional networks, batch normalisation, ReLU, better weight initialisation, Adam optimizer 3. **Hardware** -- GPU development (NVIDIA CUDA 2007), GPUs give ~10x improvement over CPUs 4. **Tools** -- Open-source frameworks (PyTorch, TensorFlow), arXiv, Kaggle 5. **Data** -- Curated datasets (MNIST, ImageNet), internet-scale data 6. **Money** -- Corporate investment attracting thousands of researchers

1.3 What is Deep Learning?

Definition: "Learning hierarchical representations from large amounts of data"

Key characteristics: - High performance on previously unsolvable problems - High computational cost requiring dedicated hardware - Dependent on optimised libraries (PyTorch, TensorFlow) - Generally hard to interpret

1.4 Deep vs Shallow Architectures

- Each layer learns a **more abstract representation** of data
- More abstract concepts are usually more **invariant** to local changes
- The last layer is a classical neural network, but the task is easier due to better representation
- All representations are learnt **end-to-end** from the objective function

Cover's Theorem: Given non-linearly separable training data, it can with high probability be transformed into linearly separable data by projecting it into a higher-dimensional space via non-linear transformation.

Cybenko's Theorem (1989): A neural network with a single hidden layer that is wide enough can represent any function. BUT: - Certain functions require exponentially many hidden units (infeasible) - The theorem does not mean the system can *learn* such a function - Deep networks may be exponentially more efficient and generalize better

1.5 Three Design Questions for Any DNN

1. **What architecture?** (layers, size, type)
2. **What loss function?**
3. **What optimisation method?**

1.6 Types of Learning Tasks

Task	Input/Output	Example Network
Supervised classification	Image -> class label	CNN
Image segmentation	Image -> pixel-wise labels	Encoder-decoder CNN
Depth estimation	Image -> continuous depth map	Encoder-decoder CNN
Unsupervised learning	Unlabelled data -> clusters/generation	Autoencoders, GANs
Reinforcement learning	States/actions -> rewards	DQN, AlphaGo

Exam Tip: The introductory lecture is rarely examined directly, but understanding the "three design questions" and Cover's/Cybenko's theorems provides useful context for longer essay-style answers.

2. MLPs, SGD & Backpropagation (Week 2)

2.1 Feed-Forward Neural Network Architecture

Components of a neuron: - **Inputs** x (vector) - **Weights** w (vector, one per input) - **Bias** b (scalar) - **Activation function** ϕ - **Output** $y = \phi(w \cdot x + b)$

General equations for a deep network (layer k):

$$\begin{aligned} z[k] &= W[k] * h[k-1] + b[k] && \text{(linear combination)} \\ h[k] &= \phi(z[k]) && \text{(activation function applied element-wise)} \end{aligned}$$

Where $W[k]$ is the weight matrix, $b[k]$ is the bias vector, and ϕ is the activation function.

2.2 Loss Functions

The **loss function** $L(f(x; w), y)$ measures how far the model prediction is from the true label:

$$\text{Total loss over dataset: } L = (1/N) * \sum_i L(f(x_i; w), y_i)$$

2.3 Gradient Descent

Update rule:

$$w = w - \alpha * \text{grad}_w(L)$$

Where α is the **learning rate** (step size).

For a vector of weights:

$$\text{grad}_w(L) = [dL/dw_1, dL/dw_2, \dots, dL/dw_N]$$

Each partial derivative uses the **chain rule**:

$$dL/dw_i = (dL/df) * (df/dw_i)$$

2.4 Stochastic Gradient Descent (SGD)

- Instead of computing gradient over ALL training points (expensive), use a **mini-batch** of samples
- Makes the descent more stochastic but much faster
- The noise from mini-batches can actually **help avoid local minima**

In high-dimensional spaces, stationary points (gradient = 0) are likely to be **saddle points** where the search can escape along at least one dimension.

2.5 Activation Functions

Function	Formula	Range	Key Properties
Step	$f(s) = 1 \text{ if } s > 0, \text{ else } -1$	$\{-1, 1\}$	Not differentiable at 0; gradient zero elsewhere; unusable for gradient descent
Sigmoid	$\text{sigma}(s) = 1/(1+e^{(-s)})$	$(0, 1)$	Smooth; saturates at extremes causing vanishing gradients
Tanh	$\text{tanh}(s) = (e^s - e^{(-s)})/(e^s + e^{(-s)})$	$(-1, 1)$	Zero-centred; still saturates
ReLU	$f(s) = \max(0, s)$	$[0, \text{inf})$	No saturation for $s > 0$; sparse activations; most commonly used

Why activation functions are needed: Without non-linear activations, stacking multiple linear layers collapses to a single linear transformation -- the network cannot learn non-linear relationships.

Exam Tip: Sigmoid derivative: $\text{sigma}'(z) = \text{sigma}(z)(1 - \text{sigma}(z))$. ReLU derivative: 1 if $z > 0$, 0 if $z < 0$, undefined at $z = 0$. You WILL be asked to compute these in exam forward/backward passes.

2.6 Backpropagation

Core idea: Compute gradients of the loss with respect to every weight in the network using the chain rule, propagating errors backward from output to input.

Four steps of backpropagation: 1. **Forward pass** -- compute all neuron values from input to output 2. **Compute loss** -- measure output error 3. **Backward pass** -- compute gradient contributions from each connection using chain rule 4. **Update weights** -- gradient descent step

Chain rule for a single neuron:

$$\begin{aligned} dL/dw_i &= (dL/dy) * (dy/dz) * (dz/dw_i) \\ &= (\text{loss derivative}) * (\text{activation derivative}) * (\text{linear part derivative}) \end{aligned}$$

For hidden layers (key formula):

$$dL/dh_j[i-1] = \text{SUM}_k \{ w_{jk}[i] * \phi'(z_k[i]) * dL/dh_k[i] \}$$

The derivative at a hidden unit is the **sum of derivatives from all neurons it connects to** in the next layer.

2.7 The Vanishing Gradient Problem

For a simple deep network with n layers:

$$dL/dw_1 = (dL/dy) * \text{product}_{\{k=2\}^n} [\phi'(z_k) * w_k]$$

- Multiplying by numbers < 1 at each layer **shrinks the gradient exponentially**

- If activation functions saturate (sigmoid/tanh), $\phi'(z) \ll 1$
- **Solutions:**
- Use **ReLU** (does not saturate for $z > 0$)
- Use **residual (skip) connections** (ResNets)
- Better **weight initialisation** (Xavier/He)

2.8 Methods of Computing Derivatives

Method	Approach	Cost
Numerical	Finite differences: $(f(x+h) - f(x))/h$	Expensive, approximate
Symbolic	Algebraic manipulation	Expression explosion
Forward-mode autodiff	Propagate derivatives forward	N forward passes for N parameters
Reverse-mode autodiff (Backprop)	Propagate derivatives backward	1 forward + 1 backward pass regardless of parameter count

Exam Pattern: Q1 almost always requires a forward pass calculation followed by a backward pass using the chain rule. Practice computing numerical values through small networks with given weights.

Key exam fact: Forward autodiff requires **P passes** (where P = number of parameters). Reverse autodiff requires **2 passes** (one forward, one backward) regardless of parameter count. This is why backpropagation is used in practice.

3. Convolutional Neural Networks (Week 3)

3.1 Why Not Fully Connected for Images?

For a 32x32x3 colour image mapped to a 6x28x28 first hidden layer:

```
Fully connected parameters = 3072 * 4704 + 4704 = 14,455,392
Conv2d parameters          = 6 * 3 * 5 * 5 + 6 = 456
```

Over **14 million** vs **456** parameters for the same transformation -- CNNs are drastically more efficient.

3.2 Key Properties of CNNs

1. **Sparse connectivity** -- kernel smaller than input; each output depends on local region only
2. **Parameter sharing** -- same kernel weights detect patterns throughout the image
3. **Translation invariance** -- $f[t[x]] = f[x]$ (classification unchanged by translation)
4. **Translation equivariance** -- $f[t[x]] = t[f[x]]$ (segmentation moves with the input)
5. **Sub-sampling (pooling)** -- concentrates maximum activations

3.3 How Convolution Works

- A **kernel/filter** (e.g., 3x3 for a colour image) slides across the entire image
- At each position, compute the **dot product** between kernel and image patch
- Produces an **activation map** (feature map)
- Multiple kernels produce multiple activation maps (channels)

PyTorch example:

```
self.conv1 = nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5)
# 6 kernels, each of shape 3x3x5 -> output: 6 activation maps
```

3.4 Output Size Formula

For input size I, filter size F, stride S, padding P:

```
Output size = floor((I + 2P - F) / S) + 1
```

Common cases: - **Half padding** (padding = (F-1)/2, stride = 1): output = same size as input - **No padding, stride 1:** output = I - F + 1 - **2x2 MaxPool, stride 2:** halves the spatial dimensions

3.5 Parameter Count for Conv Layers

```
Parameters per layer = num_kernels * (in_channels * kernel_h * kernel_w + 1)
```

The **+1** is for the bias term per kernel.

Example: Conv2d(3, 10, 5) with stride=1, padding=2: - Each kernel has $3 \times 5 \times 5 + 1 = 76$ parameters - 10 kernels: $10 \times 76 = 760$ parameters

3.6 Pooling Layers

Max Pooling (most common): - Takes the maximum value in each pooling window - Reduces spatial dimensions - Operates independently on each channel - Makes representations smaller and more manageable - Emphasises strongest activations

```
self.pool = nn.MaxPool2d(2, 2) # 2x2 window, stride 2 -> halves dimensions
```

3.7 Receptive Field

The **receptive field** is the region in the input image that affects a particular neuron.

- Grows with depth: deeper layers have larger receptive fields
- For two layers of 3x3 kernels: receptive field = 5x5
- Trend: **smaller kernels + deeper networks** (fewer parameters, hierarchical features)

3.8 1D Convolutions (Conv1d)

Used for sequential/1D data: audio signals, time series, DNA sequences.

```
nn.Conv1d(in_channels, out_channels, kernel_size)
```

Same weight-sharing and locality advantages as 2D convolutions.

3.9 Classic CNN Architectures

Architecture	Year	Key Innovation
LeNet-5	1998	First successful CNN (MNIST). [CONV-POOL-CONV-POOL-FC-FC-FC]
AlexNet	2012	Won ImageNet. Deeper, used ReLU, introduced dropout + data augmentation
GoogLeNet	2014	Won ImageNet. 22 layers with Inception modules (parallel convolutions)
VGG-16	2014	Runner-up ImageNet. 16 layers, trend of deeper + smaller kernels (3x3)
ResNet	2015	Won ImageNet. Skip/residual connections enable training 152+ layers

3.10 ResNet and Skip Connections

Problem: Very deep networks (>20 layers) see degradation -- even training error increases.

Solution: Residual connections: output = $F(x) + x$ (identity shortcut) - Allows gradient to flow directly through skip connections - Eliminates vanishing gradient problem for very deep networks - Enables training of 100+ layer networks

Exam Pattern: CNN questions appear in almost every paper. Expect to: (1) calculate output tensor shapes through conv/pool layers, (2) count parameters, (3) apply a kernel to a small image manually, (4) identify what features a kernel detects.

4. ML Workflow & Practical Aspects (Week 4)

4.1 The Eight Steps of ML Workflow

1. Define problem and assemble dataset
2. Choose a measure of success
3. Prepare the data
4. Decide on evaluation protocol
5. Monitor and diagnose training issues
6. Create a simple baseline model
7. Scale up to over-capacity model
8. Tune hyperparameters and regularise

4.2 Measures of Success

Metric	Formula / Description	When to Use
Accuracy	Correct / Total	Only for balanced datasets
Precision	$TP / (TP + FP)$	When false positives are costly
Recall	$TP / (TP + FN)$	When false negatives are costly (e.g., cancer detection)
F1 Score	$2 * (Precision * Recall) / (Precision + Recall)$	Balanced measure, but be careful -- same F1 can come from different P/R combinations
AUC-ROC	Area under ROC curve	When you need to evaluate across thresholds

Exam Tip: For medical/cancer detection scenarios: accuracy is NOT appropriate (imbalanced data), precision/recall ARE appropriate (want high recall), F1 alone is insufficient (cannot distinguish high-recall-low-precision from the reverse). This has been asked multiple times.

Why can't we use accuracy/F1 as a loss function? These metrics are **not differentiable** (or even continuous), so gradients cannot be computed. Instead, use a differentiable **surrogate loss** (e.g., cross-entropy) and hope optimising it also improves the metric.

4.3 Data Preparation

Key techniques: - **One-hot encoding** -- convert categorical features to binary vectors (avoids implying ordering) - **Binning/bucketing** -- convert continuous values to discrete bins (introduces prior knowledge) - **Normalisation** -- scale to [0,1] or [-1,1]: $x_{\text{hat}} = (x - \min) / (\max - \min)$ - **Standardisation** -- scale to mean=0, std=1: $x_{\text{hat}} = (x - \text{mean}) / \text{std}$ - **Missing data** -- imputation (mean, predicted), discard, or use out-of-range value - **Data augmentation** -- rotations, flips, noise, scaling (must be domain-appropriate)

4.4 Loss Functions by Task

Output Type	Activation	Loss Function	PyTorch
Binary classification	Sigmoid	Binary Cross-Entropy	<code>nn.BCELoss()</code> or <code>nn.BCEWithLogitsLoss()</code>
Multi-class classification	Softmax	Cross-Entropy	<code>nn.CrossEntropyLoss()</code> (includes <code>LogSoftmax</code> internally)
Regression	Linear (identity)	MSE	<code>nn.MSELoss()</code>

Exam Tip: PyTorch's `CrossEntropyLoss` assumes a **linear output layer** (no softmax) because it internally applies `LogSoftmax`. Using softmax + `CrossEntropyLoss` is a common coding mistake -- asked in 2022 paper.

Maximum Likelihood Framework: 1. Pick a known distribution to model output (e.g., Normal for regression, Bernoulli for binary classification) 2. Use the model to predict parameters of that distribution 3. Maximise the likelihood -> equivalently, minimise the **negative log-likelihood**

4.5 Evaluation Protocol

- **Train/Validation/Test split** -- typically 60/20/20 or 80/10/10
- **Validation set** -- used for hyperparameter tuning and model selection
- **Test set** -- hidden away; used ONCE for final unbiased evaluation
- **K-fold cross-validation** -- split data into K parts; train K times using different folds as validation; report mean performance. More computationally expensive but better for limited data.

Final model generation (exam favourite): 1. Use cross-validation to find best hyperparameters 2. Retrain model on ALL available data (train + validation) using best hyperparameters 3. Test ONCE on held-out test set for final score

4.6 Monitoring Training

Learning rate diagnostics: - **Too high** -- loss oscillates/explodes - **Too low** -- loss decreases very slowly, may not converge - **Just right** -- smooth, steady decrease

Underfitting vs Overfitting: - **Underfitting** -- model too simple; poor on both train and validation - **Overfitting** -- model too complex for amount of data; train loss low, validation loss high - **Solution** -- increase data, add regularisation, adjust model capacity

4.7 Hyperparameter Optimisation

Method	Description	Pros/Cons
Manual	Based on understanding of effects	Requires expertise
Grid search	Exhaustive search over defined grid	Thorough but slow; wasteful in high dimensions
Random search	Random sampling of hyperparameter space	Often more effective than grid search (Bergstra & Bengio 2012)
Bayesian optimisation	Build surrogate model (e.g., Gaussian Process) to predict performance; maximise Expected Improvement	Most sample-efficient; best for expensive evaluations

Key hyperparameters: learning rate, batch size, number of layers, neurons per layer, dropout rate, weight decay, momentum.

4.8 Class Imbalance Strategies

1. **Data augmentation** of minority class
2. **Weighted sampler** to sample classes more uniformly
3. **Weighted loss function** giving more weight to minority classes

Exam Pattern: Question 2 typically covers ML workflow -- evaluation metrics for imbalanced data, hyperparameter identification from loss curves, cross-validation procedure, and sometimes ethics.

5. Optimisation, Generalisation, Regularisation (Week 5)

5.1 Optimisation Challenges

Challenge	Description	Solution
Local minima	Many shallow minima in loss surface	SGD noise helps escape; less problematic in high dimensions
Saddle points	Local minima in some directions, maxima in others; very common in high-D	SGD naturally escapes along decreasing dimensions
Cliffs	Very steep regions from weight multiplication (especially RNNs)	Gradient clipping
Plateaus	Flat regions where gradient is very small	Momentum, adaptive learning rates
Steep valleys	Large gradient in one direction, small in another	Momentum + adaptive learning rates

5.2 SGD Properties

- Can escape local minima due to noise
- Uses all data equally (on average)
- Less computationally expensive than full-batch
- Doesn't converge in traditional sense
- **Learning rate schedule** (decreasing over time) helps convergence

Batch size considerations: - Larger batches = more accurate gradients, but diminishing returns - Small batches have **regularising effect** (noise) - GPU efficiency: power-of-2 sizes (32, 64, 128, 256) - Memory scales with batch size

5.3 Momentum

Idea: Accumulate an exponentially decaying moving average of past gradients as a "velocity."

```
v_t = beta * v_{t-1} + grad_w(L)
w = w - alpha * v_t
```

Benefits: - Accelerates through consistent gradient directions - Smooths out noisy stochastic gradients - Helps traverse steep valleys (dampens oscillations) - Gets past shallow local minima and flat regions

5.4 Adaptive Learning Rates

RMSprop:

```
s_t = gamma * s_{t-1} + (1 - gamma) * (grad)^2
w = w - alpha * grad / sqrt(s_t + epsilon)
```

- Parameters with large gradients take **small steps**
- Parameters with small gradients take **large steps**

ADAM (Adaptive Moments) -- most popular optimizer: Combines RMSprop + momentum:

```

m_t = beta1 * m_{t-1} + (1 - beta1) * grad      (1st moment: momentum)
v_t = beta2 * v_{t-1} + (1 - beta2) * grad^2    (2nd moment: adaptive weighting)
m_hat = m_t / (1 - beta1^t)                    (bias correction)
v_hat = v_t / (1 - beta2^t)                    (bias correction)
w = w - alpha * m_hat / (sqrt(v_hat) + epsilon)

```

5.5 Parameter Initialisation

- Symmetry breaking** is essential -- identical weights lead to identical gradients forever
- Too large weights -> saturation/explosion of gradients
- Too small weights -> vanishing gradients

Method	Formula	Best For
Xavier/Glorot Uniform	$W \sim U(-\sqrt{6/(m+n)}, \sqrt{6/(m+n)})$	Sigmoid, Tanh
He/Kaiming	$W \sim N(0, \sqrt{2/m})$	ReLU

Where m = fan-in, n = fan-out.

Exam Tip: The 2021 exam asked about identical weight initialisation (all 0.1). Key answer: symmetric weights produce symmetric gradients -- optimiser cannot break symmetry. Solution: random initialisation.

5.6 Bias-Variance Tradeoff

Expected Test Error = $\text{Bias}^2 + \text{Variance} + \text{Noise}$

- Noise** -- inherent uncertainty in data (Bayes error, irreducible)
- Bias** -- systematic error from model limitations (underfitting)
- Variance** -- how much model varies with different training sets (overfitting)

Reducing variance: More training data **Reducing bias:** More complex model **Tradeoff:** Increasing complexity reduces bias but increases variance

5.7 Regularisation Techniques

Regularisation = any method to reduce the generalisation gap between training and test performance.

Explicit Regularisation (adds term to loss)

L2 Regularisation (Weight Decay / Ridge):

$$L_{\text{total}} = L_{\text{data}} + \lambda * \sum(w_i^2)$$

- Favours smaller weights
- Encourages smoothness
- Does NOT force weights to zero

L1 Regularisation (LASSO):

$$L_{\text{total}} = L_{\text{data}} + \lambda * \sum(|w_i|)$$

- Encourages **sparsity** (many weights become exactly zero)
- Useful for feature selection

Implicit Regularisation

Technique	How it Works
Early stopping	Stop training before model overfits; weights don't grow large; reduces effective complexity
Data augmentation	Generate more training data via transformations; makes model robust to transformations
Dropout	Randomly drop neurons with probability p during training; forces redundancy; acts like ensemble
Ensemble methods / Bagging	Train multiple models, combine predictions; reduces variance
Adversarial training	Train on adversarial examples; improves robustness
Transfer learning	Use pre-trained model weights; leverages knowledge from larger datasets
Multi-task learning	Train on multiple related tasks simultaneously

Dropout in Detail

- At each training step, each neuron has probability p of being "dropped" (output = 0)
- At test time: no dropout, but weights are scaled by (1-p) to maintain expected activations
- **Why it works:**
 - Acts as an **ensemble** of sub-networks with shared weights
 - Neurons cannot co-adapt; become more independent
 - Forces redundancy (robustness)

Exam Pattern: Weight decay appears as a parameter in PyTorch optimizers. When asked "what does weight_decay do?" -- answer: L2 regularisation that penalises large weights to reduce overfitting.

6. Visualisation & Interpretability (Week 6)

6.1 Four Approaches to Interpreting DNNs

1. **Visualising weights/activations**
2. **Perturbation-based methods** (Occlusion, LIME)
3. **Gradient-based attribution** (Saliency, SmoothGrad, Integrated Gradients, LRP, Guided Backprop)
4. **Class Activation Mapping** (CAM, Grad-CAM)

6.2 Visualising Weights and Activations

First-layer filters: Can be visualised as images (e.g., AlexNet first layer shows edge detectors, colour blobs, frequency detectors).

Activations: Convolutional layers maintain 2D structure, so activations can be visualised as images showing which features are detected at each location.

Layers as embeddings: Use **t-SNE** or **UMAP** to project high-dimensional activations into 2D/3D to see how classes separate through the network. - Input layer: classes overlap - Middle layers: partial separation - Final layers: classes well separated

6.3 Perturbation-Based Methods

Occlusion: - Systematically occlude different parts of the input - Measure change in prediction confidence - High change = important region - Computationally expensive (one forward pass per occlusion position)

LIME (Locally Interpretable Model-Agnostic Explanations): - For a given input x , generate perturbed samples in the neighbourhood - Fit a simple interpretable model (e.g., linear) locally - The simple model explains the black-box's behaviour near x - Model-agnostic (works with any classifier)

6.4 Gradient-Based Attribution

Vanilla Gradient Saliency:

$$\text{Saliency}(x) = |dF/dx|$$

- Compute gradient of output w.r.t. input
- Magnitude = sensitivity per input dimension
- Fast (single backprop pass), architecture-agnostic
- **Limitations:** Noisy; sensitivity $\neq$ importance; gradient saturation problem

SmoothGrad: - Average gradients over N noisy copies of the input - $\text{SmoothGrad}(x) = (1/N) * \sum_i |dF/d(x + \text{noise}_i)|$ - Reduces visual noise, improves stability

Integrated Gradients (Sundararajan et al. 2017):

$$\text{IG}_i(x) = (x_i - x'_i) * \int_0^1 (dF/dx_i)(x' + a(x - x')) da$$

Where x' is a baseline (e.g., all-black image).

- Fixes saturation problem by averaging gradient along path from baseline to input
- Satisfies two critical axioms:
- **Completeness** -- attributions sum to the total output difference
- **Implementation invariance** -- same attributions for functionally equivalent networks

Key distinction: Sensitivity vs Importance - Sensitivity (vanilla gradient): how much does output *change* if input changes? (local) - Attribution (Integrated Gradients): how much does each input feature *contribute* to the output?

6.5 Layer-wise Relevance Propagation (LRP)

- Decomposes the final output score and redistributes it backward through layers
- **Conservation principle:** total relevance received by a neuron = total relevance it redistributes
- Various propagation rules (Gamma, Epsilon, Alpha-Beta) for stability
- Implemented in Captum library

6.6 Guided Backpropagation

- Modified backpropagation that only propagates positive gradients through ReLU layers
- Produces cleaner, more focused activation maps
- Combined with other methods for sharper visualisations

6.7 Class Activation Mapping (CAM / Grad-CAM)

CAM (Zhou et al. 2015): - Uses **Global Average Pooling (GAP)** layer before final classification - Weights from GAP to output indicate importance of each feature map - Produces a heatmap showing class-discriminative regions - **Limitation:** Requires GAP layer -- must retrain networks with FC layers

Grad-CAM (Selvaraju et al. 2017): - Uses **gradients** of class prediction w.r.t. feature maps to derive channel weights - No need for GAP layer -- works with any CNN architecture - Produces smoother localisation maps

```
alpha_k = (1/Z) * sum_{i,j} (dY_c / dA_k_{ij})    (importance weight for channel k)
Grad-CAM = ReLU(sum_k alpha_k * A_k)           (weighted combination of feature maps)
```

Guided Grad-CAM = Guided Backpropagation * Grad-CAM (element-wise multiplication) - Combines fine-grained detail with class-discriminative localisation

6.8 Comparison Table

Method	Type	Speed	Resolution	Class-Specific?
Vanilla Saliency	Gradient	Fast	Pixel-level	Yes
SmoothGrad	Gradient	Medium	Pixel-level	Yes
Integrated Gradients	Gradient (path)	Medium	Pixel-level	Yes
LRP	Propagation	Medium	Pixel-level	Yes
Occlusion	Perturbation	Slow	Patch-level	Yes
LIME	Perturbation	Slow	Superpixel	Yes
CAM	Activation	Fast	Low (feature map)	Yes
Grad-CAM	Gradient + Activation	Fast	Low (feature map)	Yes

Exam Pattern: Visualisation methods appear regularly (2021 Q3c, 2022 Q1e, 2023 Q3b/c). Know at least 3 methods with pros/cons. Grad-CAM is useful for detecting bias in classifiers.

7. RNNs & Transformers (Week 7)

7.1 Recurrent Neural Networks

Why RNNs? For sequential data of variable length -- text, time series, audio, video.

Key idea: Share weights across time steps. The state at time t depends on the state at time $t-1$.

Forward propagation equations (vanilla RNN):

$$h_t = \phi(W_h * h_{t-1} + W_u * x_t + b)$$

$$y_t = c + V * h_t$$

Input/Output sequence options: - Sequence -> Sequence (simultaneous): e.g., POS tagging - Sequence -> Sequence (encoder-decoder): e.g., machine translation - Sequence -> Vector: e.g., sentiment analysis - Vector -> Sequence: e.g., image captioning

7.2 Vanishing/Exploding Gradients in RNNs

Since the same unit is repeated across time steps:

$$dL/dw_1 = \text{product of } (\phi'(z_t) * W_h) \text{ across all time steps}$$

- If eigenvalues of $W_h < 1$: gradients **vanish** exponentially
- If eigenvalues of $W_h > 1$: gradients **explode**
- RNNs can handle short-term dependencies but struggle with long-term ones

7.3 LSTM (Long Short-Term Memory)

Key innovation: A **cell state** C_t that runs through the entire chain, with gates controlling information flow.

Three gates:

Gate	Formula	Purpose
Forget gate f_t	$f_t = \sigma(W_f * [h_{t-1}, x_t] + b_f)$	Decides what to remove from cell state
Input gate i_t	$i_t = \sigma(W_i * [h_{t-1}, x_t] + b_i)$	Decides what new information to store
Output gate o_t	$o_t = \sigma(W_o * [h_{t-1}, x_t] + b_o)$	Decides what to output

Cell state update:

$$C_{\tilde{t}} = \tanh(W_C * [h_{t-1}, x_t] + b_C) \quad (\text{candidate values})$$

$$C_t = f_t * C_{t-1} + i_t * C_{\tilde{t}} \quad (\text{update cell state})$$

$$h_t = o_t * \tanh(C_t) \quad (\text{output})$$

Why sigmoid for gates? Output in (0,1) acts as a soft switch (0 = block, 1 = pass). **Why tanh for candidate?** Output in (-1,1) creates candidate values (can increase or decrease cell state).

7.4 GRU (Gated Recurrent Unit)

Simplified version of LSTM with **two gates** (no separate cell state):

Gate	Formula	Purpose
Reset gate r_t	$r_t = \text{sigma}(W_r * [h_{t-1}, x_t])$	Controls how much past state to forget
Update gate z_t	$z_t = \text{sigma}(W_z * [h_{t-1}, x_t])$	Controls blend of old and new state

$h_{\text{tilde}} = \tanh(W * [r_t * h_{t-1}, x_t])$ (candidate state)
 $h_t = (1 - z_t) * h_{t-1} + z_t * h_{\text{tilde}}$ (interpolate old and new)

7.5 LSTM vs GRU Comparison

Feature	LSTM	GRU
Gates	3 (forget, input, output)	2 (reset, update)
Cell state	Separate cell state C_t	No separate cell state
Parameters	More	Fewer
Long-term memory	Generally better for very long sequences	Comparable for moderate sequences
Training speed	Slower	Faster

7.6 Limitations of RNNs

- **Sequential computation** prevents parallelisation
- Vanishing gradients with very long sequences despite LSTM/GRU
- No **explicit** modelling of long vs short-range dependencies
- Context vector becomes a bottleneck in encoder-decoder architectures

7.7 Attention Mechanism

Core idea: Instead of compressing entire input into a single context vector, allow the decoder to "attend" to all encoder states.

- Attention scores = similarity between decoder state and each encoder state
- Softmax normalisation produces attention weights
- Context vector = weighted sum of encoder states

Still has limitation: Translation remains sequential (slow training). Solved by Transformers.

7.8 Transformer Architecture (Vaswani et al. 2017)

"Attention is All You Need" -- replaces recurrence entirely with self-attention.

Key components:

1. **Tokenisation** -- words split into sub-word tokens, mapped to integer indices
2. **Positional encoding** -- since no recurrence, position info added to embeddings
3. **Multi-Head Attention (MHA)** -- enables cross-talk between token embeddings
4. **Add & Norm** -- residual connections + layer normalisation
5. **Feed-Forward** -- refines embeddings independently per position

Self-Attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q * K^T / \sqrt{d_k}) * V$$

Where: - **Q (Query)** -- representation of current word ("what am I looking for?") - **K (Key)** -- representation of all words ("what do I contain?") - **V (Value)** -- actual content to aggregate - **d_k** -- dimension of keys (scaling factor prevents large dot products)

Multi-Head Attention: - Instead of one set of (Q, K, V) matrices, learn **H parallel heads** - Each head captures different types of relationships - Outputs concatenated and mixed through a linear layer

Parameters per head: $W_Q (d_e \times d_k) + W_K (d_e \times d_k) + W_V (d_e \times d_v)$
 Total MHA parameters: $H * (d_e * d_k + d_e * d_k + d_e * d_v) + H * d_v * d_o$

Masked Self-Attention (Decoder): - Each token can only attend to previous tokens (causal masking) - Enables autoregressive training: predict next token given previous ones

Two types of attention: 1. **Self-attention** -- Q = K (same sequence attends to itself) 2. **Decoder-encoder (cross) attention** -- Q from decoder, K from encoder

7.9 Transformer vs RNN Comparison

Feature	RNN	Transformer
Parallelisation	Sequential (slow)	Fully parallel (fast)
Long-range dependencies	Struggles (vanishing gradient)	Explicit via attention
Memory	O(1) per step	O(n ²) for attention matrix
Position encoding	Implicit from sequence order	Requires explicit encoding
Training speed	Slow	Fast (parallelisable)

Exam Pattern: RNN questions appear every year. Expect: (1) forward pass through vanilla RNN with given weights, (2) computing partial derivatives (dy/dx across time steps), (3) explaining LSTM/GRU gates and why sigmoid/tanh used, (4) advantages of GRU over vanilla RNN. Transformer attention calculation appeared in 2024 paper.

8. Autoencoders (Week 8)

8.1 Overview

Autoencoder = neural network trained to reconstruct its input through a bottleneck.

Input x $\rightarrow$ Encoder $f(x)$ $\rightarrow$ Latent z $\rightarrow$ Decoder $g(z)$ $\rightarrow$ Reconstruction $\hat{x}$

Loss: Minimise reconstruction error, typically MSE: $L = ||x - \hat{x}||^2$

8.2 Why Autoencoders Are Useful

1. **Dimensionality reduction** -- latent space z has lower dimension than x (like PCA)
2. **Feature extraction** -- latent z captures important information
3. **Unsupervised pre-training** -- learn representations without labels
4. **Data denoising** -- learn to remove noise from inputs
5. **Manifold learning** -- discover low-dimensional structure in high-dimensional data

8.3 Connection to PCA

If encoder and decoder are **linear** (no activation functions):

$$\begin{aligned} h &= W_f * x && \text{(encode)} \\ \hat{x} &= W_g * h && \text{(decode)} \\ \text{Loss} &= ||W_g * W_f * x - x||^2 \end{aligned}$$

The latent space h becomes the **PCA representation** of x -- it captures the principal component directions.

Key insight: Linear autoencoder = PCA. Non-linear autoencoder = non-linear dimensionality reduction (more powerful than PCA).

Parameter/weight tying: For linear case, decoder weights can be the transpose of encoder weights: $W_g = W_f^T$ (pseudo-inverse).

8.4 Non-Linear Autoencoders

- Use fully connected layers with non-linear activations for encoder and decoder
- Can learn **non-linear manifolds** (curved lower-dimensional surfaces in high-D space)
- Related to the **manifold hypothesis**: natural data (images, text) lies on a much lower-dimensional manifold within the high-dimensional data space

8.5 Undercomplete vs Overcomplete Autoencoders

Type	Latent dim	Risk	Mitigation
Undercomplete	$\dim(z) < \dim(x)$	If encoder too complex: may encode sample i as integer i	Control encoder complexity
Overcomplete	$\dim(z) > \dim(x)$	May trivially copy x to z	Must add regularisation

8.6 Regularised Autoencoders

Sparse Autoencoders:

$$L = ||x - \hat{x}||^2 + \lambda * ||z||_1$$

- L1 penalty encourages sparse latent vectors
- Each latent dimension specialises in detecting specific features
- Often used for unsupervised pre-training of classification networks

Denoising Autoencoders: - Add noise to input: $x_{\text{noisy}} = x + \text{Gaussian noise}$ - Train to reconstruct clean x from x_{noisy} - Alternatively: use dropout on input - **Interpretation:** Learns to project noisy points back onto the data manifold - Learns a vector field pointing from off-manifold to on-manifold

Contractive Autoencoders:

$$L = ||x - \hat{x}||^2 + \lambda * ||J_f(x)||_F^2$$

Where J_f is the Jacobian of the encoder. Penalises sensitivity of encoder to input changes. - Encourages latent values to only change when necessary (along the manifold) - Equivalent to denoising autoencoder as noise $\rightarrow 0$

Stacked Autoencoders: - Train one autoencoder at a time (greedy layer-wise pre-training) - Use encoded representation as input to next autoencoder - Builds deep representations incrementally

8.7 Convolutional Autoencoders

- Use convolutional layers in encoder (downsampling) and decoder (upsampling)
- Natural for image data -- maintains spatial structure
- Decoder uses transposed convolutions (or upsampling + convolution)

Exam Tip: Know the connection between linear autoencoders and PCA. Understand why regularisation is needed for overcomplete autoencoders. Denoising autoencoders were asked about in the context of unsupervised pre-training (2021 Q3a).

9. VAEs & GANs (Week 9)

9.1 The Problem with Standard Autoencoders for Generation

- No guarantee that smooth variation in latent space produces smooth output
- "Holes" in the latent space produce garbage when decoded
- Not probabilistic -- mapping can be arbitrary as long as reconstruction is good

9.2 Probabilistic (Stochastic) Autoencoder

Instead of encoding to a single point, encode to a **distribution**:

```
Encoder outputs: mean  $\mu(x)$  and std  $\sigma(x)$ 
Sample:  $z \sim N(\mu(x), \sigma(x))$ 
Decode:  $\hat{x} = \text{decoder}(z)$ 
```

Reparameterisation trick (essential for backpropagation):

```
 $z = \mu + \sigma * \epsilon$ , where  $\epsilon \sim N(0, I)$ 
```

This makes sampling a **deterministic, differentiable** operation -- gradients can flow through.

The stochastic sampling makes the latent space smoother since similar latent values must produce similar reconstructions.

9.3 Variational Autoencoders (VAEs)

VAE = Probabilistic autoencoder + KL divergence constraint

Loss function:

```
 $L_{VAE} = \text{Reconstruction Loss} + \text{KL Divergence}$ 
 $= ||x - \hat{x}||^2 + \text{KL}(q(z|x) || N(0, I))$ 
```

The **KL divergence** term encourages the encoder's output distribution $q(z|x)$ to be close to a standard multivariate Gaussian $N(0, I)$.

KL divergence for Gaussians (closed form):

```
 $KL = -0.5 * \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2)$ 
```

Why the KL term? - Forces the latent space to follow a known distribution (multivariate Gaussian) - Allows us to **sample from $N(0, I)$** and decode to generate new data - Creates a smooth, continuous latent space with no "holes" - Prevents the encoder from collapsing each point to a distinct, isolated region

Generation with VAE: 1. Sample z from $N(0, I)$ 2. Pass z through decoder to generate new data x

Concept vectors: By interpolating in the latent space, can smoothly morph between different data characteristics (e.g., add "smile" vector to face).

9.4 Generative Adversarial Networks (GANs)

Completely different approach from VAEs -- no explicit probabilistic model.

Two networks: 1. **Generator G** -- takes random noise $z \sim N(0, I)$ and produces "fake" data $G(z)$ 2. **Discriminator D** -- predicts whether input is "real" (from training data) or "fake" (from G)

Training objective (minimax game):

$$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$$

Where: - G tries to minimise (fool the discriminator) - D tries to maximise (correctly classify real vs fake) - $x \sim p_{\text{data}}(x)$ -- real data - $z \sim p_z(z)$ -- random noise (typically Gaussian) - $D(x)$ -- probability that x is real (want close to 1 for real data) - $D(G(z))$ -- probability that generated data is real (D wants close to 0, G wants close to 1)

9.5 GAN Training Details

Problem with original generator loss: $\min_G \log(1 - D(G(z)))$ - Initially, generated images are bad, so $D(G(z))$ is close to 0 - But the gradient of $\log(1 - D(G(z)))$ is very small near $D(G(z)) = 0$ - Learning is extremely slow when it matters most!

Better generator objective: $\max_G \log(D(G(z)))$ or equivalently $\min_G -\log(D(G(z)))$ - Maximise likelihood of discriminator being **wrong** - Gradient is large when $D(G(z))$ is near 0 (where training needs to happen)

DCGAN (Deep Convolutional GAN): - Uses convolutional architecture for both generator and discriminator - Generator uses transposed convolutions to upsample noise to images - Key architectural guidelines: batch normalisation, ReLU in generator, LeakyReLU in discriminator

Equilibrium: Training converges when G produces data indistinguishable from real data and D outputs 0.5 for everything (pure guessing).

9.6 CycleGAN (Zhu et al. 2017)

Purpose: Unpaired image-to-image translation (e.g., horse -> zebra, photo -> painting).

Architecture: Two GANs linked in a cycle: - G_{AB} : translates domain A -> domain B - G_{BA} : translates domain B -> domain A - **Cycle consistency loss:** $G_{BA}(G_{AB}(x_A))$ should approximately equal x_A

9.7 VAE vs GAN Comparison

Feature	VAE	GAN
Approach	Probabilistic model + variational inference	Game-theoretic adversarial training
Latent space	Structured (forced to Gaussian)	Unstructured
Training	Stable (single loss)	Often unstable (two competing networks)
Output quality	Tends to be blurry	Sharper, more realistic
Likelihood	Can compute (approximately)	Cannot compute $p(x)$
Mode coverage	Good (covers full distribution)	May suffer mode collapse
Interpolation	Smooth, meaningful	Can be meaningful (with DCGAN latent arithmetic)

Exam Pattern: GANs appeared in the 2025 paper Q2b -- describing architecture parts, meaning of terms in the minimax equation. Know the GAN equation terms thoroughly. VAE questions may ask about the reparameterisation trick, KL divergence purpose, and how to generate new samples.

10. Exam Strategy & Past Paper Analysis

10.1 Exam Format

- **Duration:** 1 hour 30 minutes + 30 minutes additional
- **Structure:** 3 compulsory questions, 60 marks total
- **Weighting:** Q1 typically 20 marks, Q2 typically 20 marks, Q3 typically 20 marks
- **Open book** (since 2021)
- **Calculator:** Not required but numerical calculations expected

10.2 Question Patterns Across All Years

Topic	2021	2022	2023	2024	2025
Forward/backward pass calculation	Q1	Q3(RNN)	Q2c-g	Q2c-f	Q3a(RNN)
Activation functions (why sigmoid/ReLU)	Q1a	Q3e	--	--	Q1a
Parameter counting	--	Q1b, Q2a	Q1b,c	Q1e	Q1c
Output tensor shapes	--	Q2a	Q1d	Q1d	--
CNN convolution/pooling calculation	--	Q2c,d	Q2a,b	Q2a,b	Q2a
Loss function choice	Q2b	Q1f	--	Q1f	Q1b
Evaluation metrics (accuracy/precision/recall)	Q2a	--	--	--	--
Hyperparameter identification from curves	Q2c,d	--	--	--	Q1d,e
Learning rate / momentum / weight decay	Q2c	Q1c,d	--	--	Q1d
Cross-validation procedure	Q2e	--	--	--	--
Visualisation methods	Q3c	Q1e	Q3b,c	--	--
RNN forward pass	--	Q3a	--	Q3a	Q3a
RNN gradient computation	--	Q3b,c	--	--	Q3a(ii)
LSTM/GRU gates explanation	--	Q3e,f	--	--	--
Attention/Transformer	--	--	Q3b	Q3 (full)	Q3b,c
Autoencoders / unsupervised pre-training	Q3a	--	--	--	--
GANs	--	--	--	--	Q2b
Ethics	Q3d	Q2f	Q3d,e	--	--
Autodiff (forward vs reverse)	Q1b	--	--	--	--
Conv1d	--	--	--	Q1,Q2a	--
PyTorch code analysis	--	Q1(all)	Q1(all)	Q1(all)	Q1(all)

10.3 Most Frequently Tested Topics (Priority Order)

1. **Forward and backward pass calculations** -- appears EVERY year
2. **Parameter counting** -- appears almost every year
3. **CNN operations** (convolution, pooling, tensor shapes) -- appears almost every year
4. **Loss function and activation function choice** -- very frequent
5. **RNN forward pass and gradients** -- appears frequently
6. **Hyperparameter effects** (learning rate, batch size, etc.) -- frequent
7. **Visualisation/interpretability methods** -- appears most years
8. **Attention/Transformers** -- increasing frequency (2024, 2025)
9. **Ethics** -- appears some years, usually as part of Q3

10. **GANs/VAEs** -- appeared in 2025, likely to recur

10.4 Exam Tips by Question Type

Q1: PyTorch Model Analysis (typically ~20 marks) - Read the PyTorch code carefully -- identify layer types, input/output dimensions - Count parameters: Linear(m, n) has $m \cdot n + n$ parameters; Conv2d(c_in, c_out, k) has $c_{out} \cdot (c_{in} \cdot k \cdot k + 1)$ parameters - Track tensor shapes through each layer using the output size formula - Check for coding mistakes (e.g., softmax before CrossEntropyLoss)

Q2: Computation Questions (~20 marks) - Practice manual convolution on small images with specific kernels - Know how to apply max pooling - For backpropagation: write the chain rule step by step, substitute numerical values - For matrix notation: be comfortable with $z = Mx + b$, $h = \text{sigma}(z)$, $y = Nh$

Q3: Conceptual/Applied Questions (~20 marks) - Often involves RNNs, attention, or applied scenarios - Ethics questions: cover bias in training data, impact on groups, feedback loops, explainability - Comparison questions: prepare concise tables (RNN vs Transformer, LSTM vs GRU, VAE vs GAN)

10.5 Key Formulas to Know Cold

Output size (conv/pool):	$\text{floor}((I + 2P - F) / S) + 1$
SGD update:	$w = w - \alpha * \text{grad}(L)$
Momentum:	$v = \beta * v_{\text{prev}} + \text{grad}(L)$ $w = w - \alpha * v$
Sigmoid:	$\text{sigma}(z) = 1 / (1 + e^{-z})$
Sigmoid derivative:	$\text{sigma}'(z) = \text{sigma}(z) * (1 - \text{sigma}(z))$
ReLU:	$f(z) = \max(0, z)$
ReLU derivative:	$f'(z) = 1 \text{ if } z > 0, \text{ else } 0$
Chain rule (backprop):	$dL/dw = (dL/dy) * (dy/dz) * (dz/dw)$
Hidden layer backprop:	$dL/dh_j = \sum_k \{ w_{jk} * \phi'(z_k) * dL/dh_k \}$
Self-attention:	$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$
VAE loss:	$L = x - \hat{x} ^2 + KL(q(z x) N(0,I))$
GAN objective:	$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$
L2 regularisation:	$L_{\text{total}} = L_{\text{data}} + \lambda * \sum(w_i^2)$
F1 score:	$F1 = 2 * P * R / (P + R)$
Linear params:	$m * n + n$ (weight matrix + bias)
Conv2d params:	$c_{out} * (c_{in} * k_h * k_w + 1)$

10.6 Common Exam Traps

1. **CrossEntropyLoss in PyTorch** expects raw logits (linear output), NOT softmax output
2. **Accuracy** is inappropriate for imbalanced datasets
3. **F1 score** cannot distinguish high-precision-low-recall from high-recall-low-precision
4. **Forward autodiff** requires P passes (one per parameter); **Reverse autodiff** requires 2 passes total
5. **Weight decay** in PyTorch optimizer = L2 regularisation (not just reducing weights)
6. **Symmetric weight initialisation** prevents learning (cannot break symmetry)
7. **ReLU derivative** is undefined at exactly 0 (conventionally set to 0)
8. **Dropout** scales weights at test time (or equivalently scales during training with inverted dropout)
9. **Max pooling** output size follows same formula as convolution
10. **Half padding** = padding of $(F-1)/2$ keeps spatial dimensions the same (stride 1)

10.7 Time Management

- **20 minutes per question** (with buffer time for review)
- Start with the question you are most confident about
- For calculation questions: show ALL working -- partial credit is awarded
- For essay questions: use structured bullet points, not long paragraphs
- If stuck on a numerical calculation, state the formula and move on

10.8 Sample Answer Framework for Ethics Questions

Cover these four pillars: 1. **Bias in training data** -- historical prejudices, demographic imbalance 2. **Impact on specific groups** -- underrepresented groups get less reliable predictions 3. **Feedback loops** -- biased decisions create biased future training data 4. **Explainability** -- black-box models cannot justify decisions; required for high-stakes applications

End of COMPSCI5103 Deep Learning Exam Notes. Good luck!